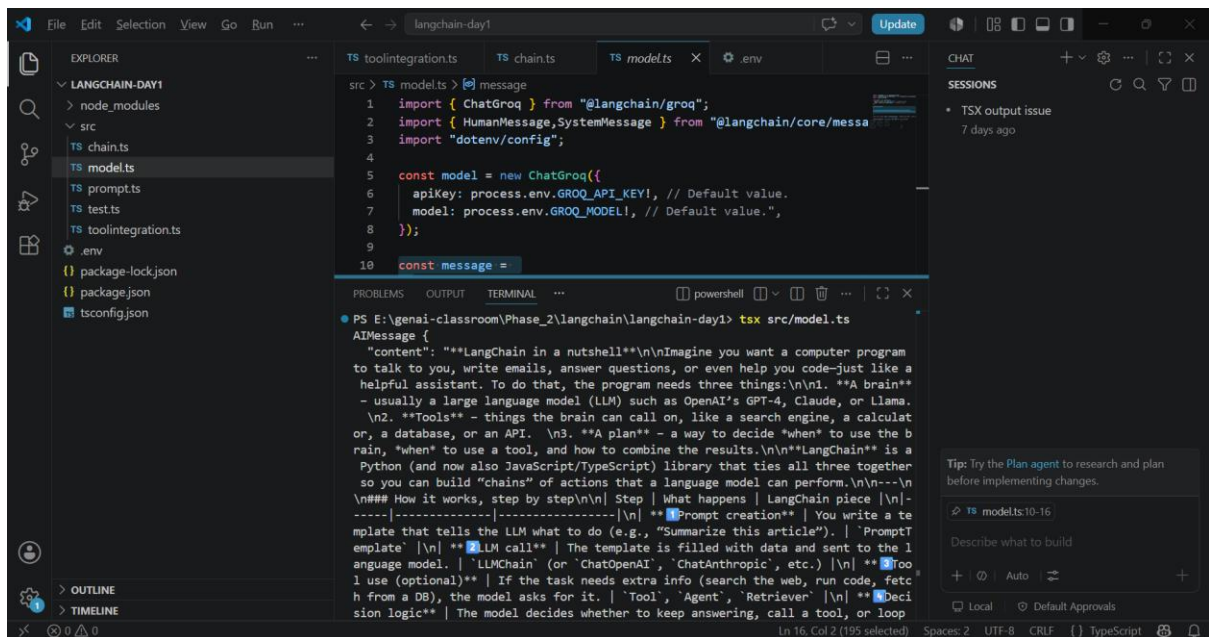
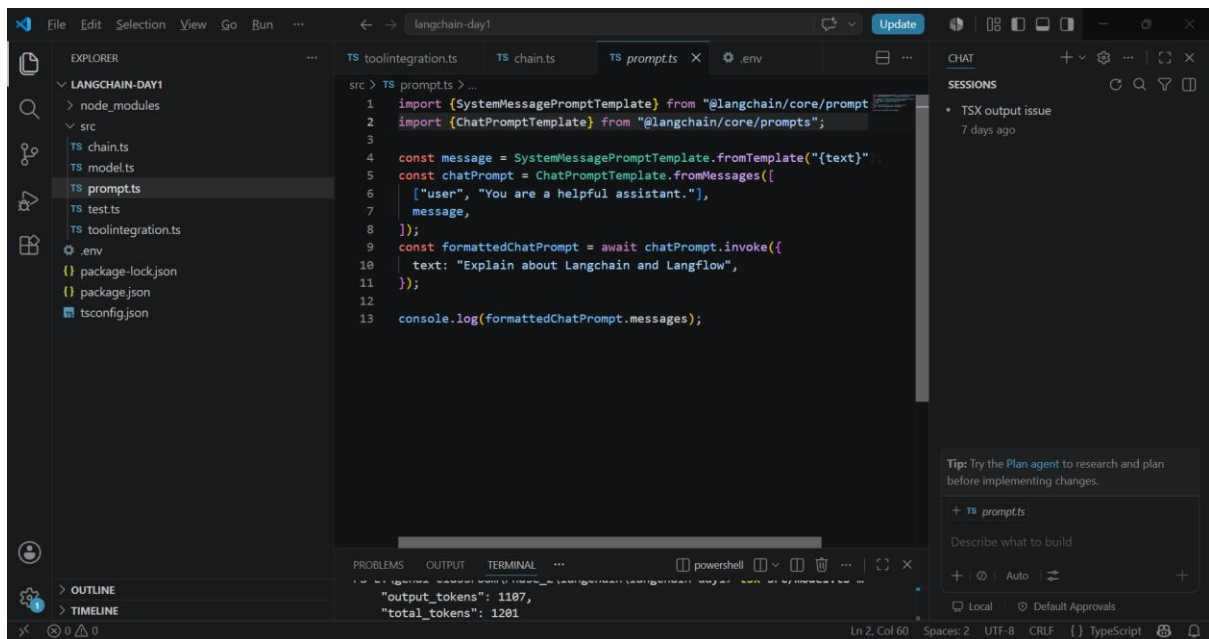


models.ts:



prompts.ts:



```
1 import {SystemMessagePromptTemplate} from "@langchain/core/prompts";
2 import {ChatPromptTemplate} from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6   ["user", "You are a helpful assistant."],
7   message,
8 ]);
9 const formattedChatPrompt = await chatPrompt.invoke({
10   text: "Explain about Langchain and Langflow",
11 });
12
13 console.log(formattedChatPrompt.messages);
```

PROBLEMS OUTPUT TERMINAL

output_tokens: 1107,
total_tokens: 1201

CHAT

SESSIONS

- TSX output issue
7 days ago

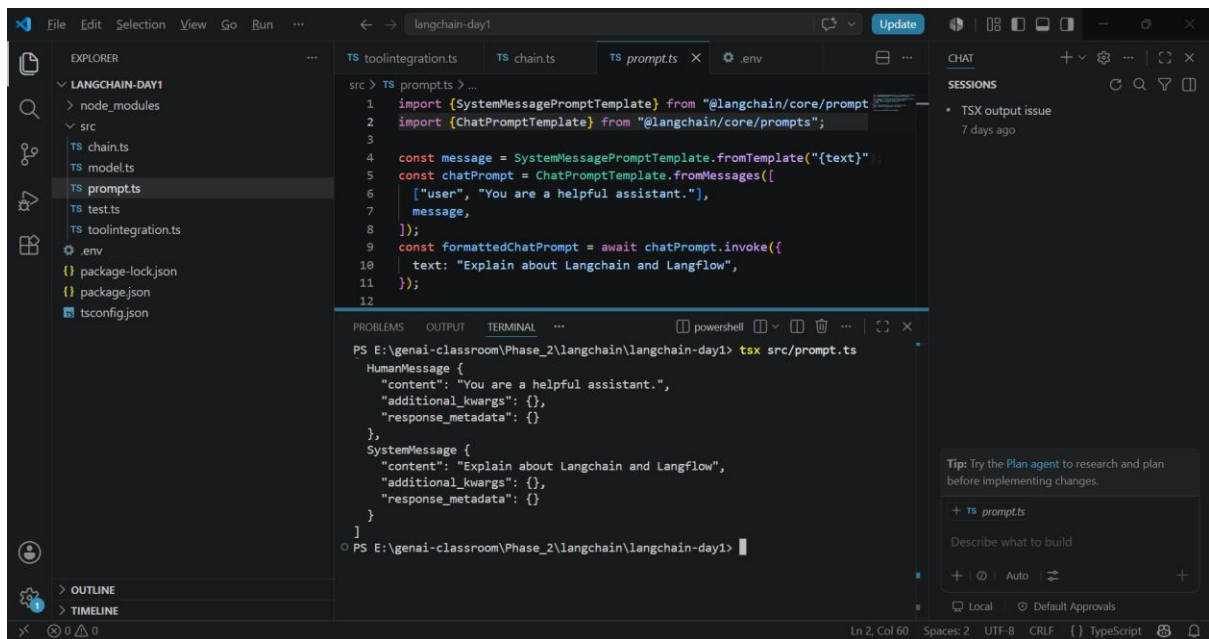
Tip: Try the Plan agent to research and plan before implementing changes.

+ TS prompts

Describe what to build

+ | | Auto | +

Local Default Approvals



```
1 import {SystemMessagePromptTemplate} from "@langchain/core/prompts";
2 import {ChatPromptTemplate} from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6   ["user", "You are a helpful assistant."],
7   message,
8 ]);
9 const formattedChatPrompt = await chatPrompt.invoke({
10   text: "Explain about Langchain and Langflow",
11 });
12
```

PROBLEMS OUTPUT TERMINAL

PS E:\genai-classroom\Phase_2\langchain\langchain-day1> tsx src/prompts.ts

```
{
  HumanMessage {
    "content": "You are a helpful assistant.",
    "additional_kwargs": {},
    "response_metadata": {}
  },
  SystemMessage {
    "content": "Explain about Langchain and Langflow",
    "additional_kwargs": {},
    "response_metadata": {}
  }
}
```

PS E:\genai-classroom\Phase_2\langchain\langchain-day1>

CHAT

SESSIONS

- TSX output issue
7 days ago

Tip: Try the Plan agent to research and plan before implementing changes.

+ TS prompts

Describe what to build

+ | | Auto | +

Local Default Approvals

toolintegration.ts:

```
src > TS toolintegration.ts > ...
1 import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";
2 import { MistralAIEmbeddings } from "@langchain/mistralai";
3 import { MongoClient } from "mongodb";
4 import "dotenv/config";
5 import { setServers } from "dns";
6 setServers(["8.8.8.8", "8.8.4.4"]);
7
8 const client = new MongoClient(process.env.MONGODB_ATLAS_URI!);
9 const collection = client
10   .db(process.env.MONGODB_ATLAS_DB_NAME!)
11   .collection(process.env.MONGODB_ATLAS_COLLECTION_NAME!);
12
13 const embeddings = new MistralAIEmbeddings({
14   model: "mistral-embed",
15 });
16
17 const vectorStore = new MongoDBAtlasVectorSearch(embeddings, {
18   collection,
19   indexName: "vector_index_testcases", // Defaults to "default"
20   textKey: "text", // Defaults to "text"
21   embeddingKey: "embedding" // Defaults to "embedding"
22 });
23
24 const similaritySearchWithScoreResults = await vectorStore.similaritySearchWithScoreResults(
25   "test",
26   0.5
27 );
```

```
PS E:\genai-classroom\Phase_2\langchain\langchain-day1> tsx src/toolintegration.ts
Document {
  pageContent: '',
  metadata: {
    _id: new ObjectId('6a378a6765672f302484d685'),
    module: 'IP Billing',
    id: 'TC_3342',
    prerequisites: 'UHID:AP31.0003504849',
    title: 'Dashboards ->IP Dash Board->Generate IP Bill',
    description: 'Verify user able to generate Generate IP Bill',
    steps: '1.Launch the Application.\r\n' +
      '2.Login to the application with valid credentials\r\n' +
      '3.Click on Common Support Services Tab and select Billing from the drop do\r\n' +
      'wn.\r\n' +
      '4.Navigate to Functions -> Dashboards -> IP Dash Board.\r\n' +
      '5.Click on IP Dash Board.\r\n' +
      '6.Enter UHID no and click on "View Details".\r\n' +
      '7.Right click on IP Number and click on "Generate IP Bill".\r\n' +
      '8.Data is automatically populated in fields like Bill Details, Deposit Det\r\n' +
      'ails, Discount Details, Packages, Deposit Relaxation, Previous Interim Bill, Cre\r\n' +
      'dit Details, Tax Details and Adjustment.\r\n' +
      '9.If required to view Bed Details click on Bed Details and click on BedCat
```

chain.ts:

```
1 import { ChatGrok } from "@langchain/groq";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parser";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7   ["system", "Please review the user query below and determine if it contains any sensitive information."],
8   ["user", "{template}"]
9 ]);
10
11 const chatModel = new ChatGrok({
12   apiKey: process.env.GROQ_API_KEY!, // Default value.
13   model: process.env.GROQ_MODEL!, // Default value.",
14 });
15 const outputParser = new StringOutputParser();
16 const chain = prompt.pipe(chatModel).pipe(outputParser);
17
18 const res = await chain.invoke({ template: "I hate you" });
19
20 console.log(res);
21
22
23
24
```

steps: '1.Launch the Application.\n' +
'2.Login to the application with valid credentials\n' +

```
1 import { ChatGrok } from "@langchain/groq";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parser";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7   ["system", "Please review the user query below and determine if it contains any sensitive information."],
8   ["user", "{template}"]
9 ]);
10
11 const chatModel = new ChatGrok({
```

PS E:\genai-classroom\Phase_2\langchain\langchain-day1> tsx src/chain.ts
Toxic
PS E:\genai-classroom\Phase_2\langchain\langchain-day1>

